ENGINEERING WALKTHROUGH

JeevanSetu, end to end

What every part of the codebase does, why it was built that way, and exactly where the honest limits are.

REPOSITORY	**citibank2.0, branch main**
BACKEND	**FastAPI · Python 3.12+ · SQLite**
FRONTEND	**Next.js 16 · React 19 · Tailwind 4**
AI	**Gemini Developer API, free tier**
TESTS	**128 backend · 19 frontend**

HOW TO READ THIS

Sections 1 to 4 explain the shape of the system. Sections 5 to 8 are the parts that decide anything, and they are the ones worth understanding properly. Sections 9 to 13 cover running it, testing it, and everything that is deliberately unfinished.

Contents

1. What the product actually does

2. The one architectural rule that shapes everything

3. System map

4. The data model

5. Step 1 — upload and validation

6. Step 2 — AI extraction

7. Step 3 — deterministic detection

8. Step 4 — scoring, explanations, and the UI

9. Simulated fixes and undo

10. The rule set and its sources

11. Running, seeding, and demoing it

12. Tests, and what they actually guarantee

13. Deployment, and what is deliberately unfinished

1. What the product actually does

A person's assets are promised to people twice, in two places that never talk to each other. A bank nomination form says one name. A will says another. Both are valid documents; nothing in either one points at the other.

JeevanSetu takes a set of documents belonging to one person, extracts who is named for which asset in each of them, and cross-references them against a small set of rules drawn from Indian primary sources. When two documents disagree about the same asset, it reports a *review conflict*: both quotes side by side, the rule that fired, what the rule can and cannot conclude, and a suggested next step.

It never says who inherits. That distinction runs through the whole codebase and is the reason the design looks the way it does.

The three kinds of output

FINDING TYPE	MEANS	SCORE COST
`review_conflict`	Two documents name different people for the same asset under an approved rule.	25 or 10
`validity_warning`	One document's own status is doubtful — for example an NPS nomination that appears to predate a marriage with no confirmed replacement.	25 or 10
`scope_gap`	The engine met a case the rules deliberately exclude. Labelled "manual review required". Never a conflict, never above low severity.	3

2. The one architectural rule that shapes everything

> **THE BOUNDARY**
> AI may read documents and may reword a finding. AI may never decide whether a rule matched. Every conflict decision is plain Python.

This is enforced, not just intended. `backend/app/detection.py` imports no model client at all. Two tests hold that line: one greps the module source and fails if it contains any reference to the AI SDK; the other constructs a poisoned environment so that creating a model client during `detect()` would raise, then runs a full detection to prove nothing tries.

Concretely, AI is used in exactly two places:

- **Extraction.** Given a document, return structured fields plus the exact source text each value came from.
- **Explanation.** Given a finding the engine has *already decided*, write two short paragraphs a non-lawyer can read. If this call fails, the finding still appears with its deterministic summary.

Because of this split, every result on screen can be traced to a rule ID and a quoted sentence. There is no step where the answer came from a model's judgment.

3. System map

```
Browser (Next.js, :3000)
  │   app/page.tsx                    persona list
  │   app/personas/[id]/page.tsx      results, score, vault
  │   app/personas/[id]/estate-case   concept page (clearly labelled)
  │   app/api/backend/[...path]       server-side proxy, for hosting only
  ▼
FastAPI (:8000)
  ├── main.py          app wiring, CORS, optional API token, /health
  ├── api.py           all routes, orchestration, error mapping
  ├── uploads.py       validation and generated storage keys
  ├── extraction.py    Gemini classification + structured extraction
  ├── detection.py     the 7 rules and the gap markers   ← no AI, ever
  ├── scoring.py       readiness score + completeness gaps
  ├── explanations.py  Gemini wording for decided findings
  ├── estate_case.py   the labelled concept view
  ├── repository.py    every SQL statement lives here
  ├── db.py            connection, schema version guard
  └── SQLite           backend/data/jeevansetu.db
```

The layering is strict in one direction: routes call the repository, the repository owns all SQL, and the rule engine is a pure function that receives a list of records and returns a list of findings. The engine never touches the database, the network, or the clock.

4. The data model

Six tables. Schema version 4. There is no migration framework: on a version change the backend refuses to start and tells you to delete the synthetic database, which is acceptable precisely because all the data is disposable.

TABLE	HOLDS
`personas`	One synthetic individual. The unit everything hangs off.
`documents`	One upload: filename, MIME type, generated storage key, SHA-256 of the content, classified type, document date, status, and whether extraction was live or fallback.
`extracted_fields`	One named person, for one asset, under one mechanism — plus the exact source text and locator it came from. 18 columns per row.
`field_edits`	The log of simulated fixes: previous value, new value, and whether it was undone.
`detection_runs`	One detection run: the rules version used and the resulting score.
`conflicts`	The findings of a run, conflicts and gaps alike, distinguished by `finding_type`.

The field that does the real work

An `extracted_fields` row is not just "Kavya is the nominee". It carries every qualifier the rules need in order to stay inside their documented scope:

COLUMN	WHY A RULE NEEDS IT
`asset_reference`	Two documents are only compared when they point at the same asset. No reference means no comparison, and a gap instead.
`mechanism_type`	Separates a nomination from a will bequest. The comparison is always one against the other.
`registration_status`, `registration_date`	Possession of a form does not prove the institution recorded it. Only a confirmed registration with a real date can supersede another record.
`policy_kind`, `mwpa_section_6_applies`, `assignment_status`	The insurance rules apply only to a life policy that is not under the Married Women's Property Act and has no active assignment.
`holding_pattern`	The bank and mutual-fund rules are scoped to single-holder assets. A joint asset leaves rule scope.
`source_text`, `source_locator`	Every value on screen can be traced to a sentence and a page or clause. This is what makes a finding checkable.

5. Step 1 — upload and validation

`POST /personas/{id}/documents`, handled by `uploads.py`.

- **Synthetic attestation is mandatory.** The request must carry `synthetic_confirmed=true` or it is rejected with `synthetic_confirmation_required`. This is a product decision, not paperwork: the Gemini free tier permits Google to use submitted content to improve its products, so real documents must never enter.
- **Four independent checks.** Extension allowlist (`.pdf .png .jpg .jpeg .webp .txt`), declared MIME type matching that extension, a size ceiling of 10 MB, and a SHA-256 of the content for identity.
- **Generated storage keys.** The file is never written under its uploaded name, so a hostile filename cannot influence the path.

6. Step 2 — AI extraction

`extraction.py` makes one multimodal Gemini call per document that both classifies the document and extracts every named person in it.

- **Structured output, not parsed prose.** The call passes an explicit JSON response schema, so the model returns typed fields rather than text that has to be interpreted. The schema is hand-written and flat, because handing the SDK a nested Pydantic model made it emit a key the API rejected.
- **Source text is required per value.** The prompt requires the exact sentence and a locator for every field. A value with no evidence is not useful, because the whole interface is built on showing the user what the document said.
- **Unknown is a first-class answer.** Qualifiers use explicit `unknown` values rather than being guessed. The rules then decide what to do with uncertainty, which is a deterministic decision, rather than the model deciding by filling in a blank.
- **Failure is visible and recoverable.** A provider error keeps the upload, marks the document `failed` with an error code, and leaves it retryable. It never silently produces empty fields.
- **Prompt-injection hardening.** Document text is wrapped in delimiters and any attempt inside the document to close those delimiters is stripped, so a sentence in an uploaded file cannot pose as an instruction.

7. Step 3 — deterministic detection

`detection.py` is a pure function: records in, findings out. It is the heart of the project and contains no AI, no I/O, and no randomness.

Fixed order, because order changes the answer

```
1. NPS-VALIDITY-001            a document's own validity comes first
2. LIFE-NOTICE-001             then the notice-to-insurer question
3. RECORD-SUPERSESSION-001     then which record the institution holds
4. BANK / LIFE / MF / NPS will-mismatch rules
5. Scope gaps (GAP-*)          everything the rules exclude
6. finalize()                  dedupe, then order by severity
```

The ordering exists so findings suppress each other correctly. If an NPS nomination looks invalid on its own terms, saying "it also disagrees with the will" is noise, so the will-mismatch rule is suppressed for that group and the validity warning is shown instead. Deduplication is by rule, asset and the unordered pair of sources, so the same disagreement cannot be reported twice.

Matching two documents is deliberately conservative

- **Same asset** requires the institution and normalised asset reference to agree. A near-match — a truncated reference, an institution-name variant — produces `GAP-ASSET-NEAR-MATCH` rather than being treated as either the same asset or a clean result.
- **Different people** is a name comparison after normalisation, and a confirmed alias can resolve it. A spelling variation alone never proves two different people.
- **Nothing is inferred from dates.** "The newest document wins" is written down as an explicit non-rule. A later file date proves nothing; only a confirmed registration date with the institution can establish supersession.

Gap markers: the mechanism that keeps it honest

When the engine meets a case the rules exclude, it does not force it through the nearest rule and it does not stay silent. It emits a `GAP-` marker: severity `low`, type `scope_gap`, displayed as "manual review required". The prefix exists so a gap can never be mistaken for an approved rule ID.

MARKER	FIRES WHEN
`GAP-JOINT-HOLDING`	The asset is held jointly, so the single-holder rules do not apply.
`GAP-LIFE-SCOPE`	An insurance nomination is not a confirmed plain life policy.
`GAP-NPS-RELATIONSHIP`	A nominee's relationship is neither clearly inside nor clearly outside the regulation's family definition.
`GAP-ASSET-REFERENCE`	A record has no asset reference, so matching is impossible.
`GAP-ASSET-NEAR-MATCH`	Two records nearly match on the asset, but not exactly.
`GAP-UNSUPPORTED-SUPERSESSION`	Two records disagree on an asset outside supersession scope.
`GAP-MISSING-NOMINATION`	The will assigns an asset with no nomination record uploaded at all.

A test enforces this: it greps the engine, the scoring module and the routes for `GAP-` strings and fails if any of them is missing from `RULES.md`. An undocumented marker is treated as an undocumented rule.

8. Step 4 — scoring, explanations, and the UI

The Continuity Readiness Score

```
score = 100 − (25 × high) − (10 × medium) − (3 × gap),  clamped to 0…100
```

Deliberately simple arithmetic, shown on screen with its own breakdown, so a user can see why the number moved. The breakdown separates conflicts from gaps by *finding type*, not by severity — an earlier version partitioned by severity and displayed validity warnings under the conflicts heading, which review caught.

> **WORTH UNDERSTANDING**
>
> Leaving rule scope *raises* the score: a gap costs 3 where a conflict costs 25. The same joint bank account that would score 75 as a conflict scores 97 as a gap. That is why the score is labelled "review priority for synthetic documents, not legal readiness" everywhere it appears, and why a gap card says manual review required rather than anything reassuring.

A run recorded before scoring existed has a null score. The UI states that explicitly and invites a re-run, rather than rendering it as a red *0 / 100* — an early version did exactly that, next to a green "no conflicts found" panel.

Explanations

For each decided finding, Gemini is asked for a plain-language paragraph and a suggested fix. Four constraints:

- At most 10 findings per run get a model explanation, bounding free-tier usage.
- The finding is already decided; the model is told the outcome, never asked for it.
- Wording containing a legal conclusion is rejected outright by a deterministic phrase check — "legally owns", "rightful owner", "overrides the will", "is the legal heir" and similar.
- If the call fails or is rejected, the card shows its deterministic summary and says the plain-language wording is unavailable. Scope gaps never show that notice, because they were never meant to have model wording.

What the interface guarantees

- Every finding card carries the rule ID, the scope note and the disclaimer.
- Conflicts and gaps are in separate sections with separate headings.
- The vault shows the original extracted quote beside every value, so an edited value can never be mistaken for the document's own text.
- Fallback seed data is labelled "pre-verified fallback data, not a live AI extraction". Live extraction names the model that produced it.
- The score card is marked stale after an edit until detection is re-run, so a number can never quietly describe an older state.

9. Simulated fixes and undo

The vault lets you change a named person, which is what makes the demo interactive: fix the conflict, re-run, watch the score move.

- **Only the extracted record changes.** Never the uploaded document, never an institution's record. The UI says so.
- **Every real edit is logged** with its previous and new value. `POST /fields/{id}/undo` reverts the most recent edit that has not been undone; edits unwind one at a time, and the "edited" badge clears only when none remain.
- **Saving an unchanged value is not an edit.** Otherwise the interface would claim a change that never happened.
- **Re-extraction protects your edits.** `POST /documents/{id}/extract` returns 409 if it would discard user edits, unless you pass `force=true`. A forced re-extraction replaces the document's fields and removes their edit history with them — that boundary is documented rather than glossed over.

10. The rule set and its sources

Seven rules, version `1.0.0-candidate`, each with a predicate, a severity, its exclusions and worked examples in `backend/rules/RULES.md`. Sources were re-verified on 29 September 2026.

RULE	PRIMARY SOURCE
`BANK-WILL-001` Bank nominee differs from will recipient	Supreme Court of India, *Ram Chander Talwar v. Devender Kumar Talwar*, applying Banking Regulation Act s. 45ZA
`LIFE-NOTICE-001` Will changes a life nomination but insurer notice is unconfirmed	Insurance Act 1938, s. 39(2)–(3)
`LIFE-WILL-001` Life nominee differs from will recipient	Insurance Act 1938, s. 39(6)–(7), with s. 38 and s. 39(12) as exclusions
`MF-WILL-001` Folio nominee differs from will recipient	SEBI circular of 28 February 2025 and its Annexure A nomination form
`NPS-WILL-001` Valid-looking NPS nominee differs from will recipient	PFRDA (Exits and Withdrawals under the NPS) Regulations 2015, Chapter VII
`NPS-VALIDITY-001` NPS nomination appears invalid after marriage	Same PFRDA regulation, nomination provisos (iv)–(vi)
`RECORD-SUPERSESSION-001` Conflicting records with no proven supersession chain	The registration provisions across all four sources above

What the rules explicitly refuse to say

`RULES.md` section 9 lists these as non-rules, and tests assert the engine never produces them:

- "The newest dated document automatically wins."
- "A nominee always owns the asset", or "a will always overrides a nominee", or the reverse.
- Who is a legal heir, or whether any document, signature, marriage or relationship is valid.
- That holding a nomination form proves the institution registered it.
- That a missing nomination is a conflict — it is a completeness gap.
- Anything about joint accounts, lockers, general insurance, demat shares, trusts, tax, creditors or foreign assets.
- Anything drawn from the March 2026 SEBI consultation draft, because it is still a draft.

11. Running, seeding, and demoing it

```
cp .env.example .env          # GEMINI_API_KEY optional for the default demo
docker compose up -d --build
docker compose exec backend python scripts/seed_demo.py --yes
```

Then open `http://localhost:3000`. Both ports bind to loopback only.

Two seed modes, and why the distinction matters

MODE	WHAT IT DOES
fallback default	Inserts a pre-verified snapshot of earlier live extraction output and labels it `manual_fallback`. No API key, no network call, about a second. The UI says the data is not a live extraction.
live `--mode live`	Uploads all seven synthetic PDFs and extracts each one with Gemini. Needs a key, takes about 35 seconds, retries on provider errors.

The snapshot is never hand-edited. `scripts/export_demo_fallback.py` regenerates it from a live-seeded database and refuses to export anything that is not live extraction output or that contains a user edit — otherwise the `manual_fallback` label would be a lie.

The two personas

PERSONA	DOCUMENTS	SCORE
Aarav Demo-Mehta the conflict	Will, bank nomination, insurance nomination, mutual fund nomination. The will leaves account `SB-DEMO-4821` to Anika; the bank form names Kavya. The insurance and mutual fund records agree with the will.	75
Rohan Demo-Sen the clean contrast	Will, bank nomination, insurance nomination, all naming the same person.	100

Exactly one conflict fires for Aarav: `BANK-WILL-001`. That is by design — one unambiguous finding is more convincing in a four-minute demo than five noisy ones.

12. Tests, and what they actually guarantee

128 backend tests and 19 frontend tests. Counts are not the point; what they hold is. The interesting ones enforce the promises rather than the code:

- **The AI boundary.** `detection.py` contains no model reference, and constructing a client during a detection run fails.
- **Rules match their document.** Summaries, scope notes and suggested actions are compared against `RULES.md`, so code and documentation cannot drift apart.
- **Every gap marker is documented.** A marker in the code that is missing from `RULES.md` fails the suite.

- **Each rule has a positive and a negative case**, asserting the rule ID, severity, both sources and the exact disclaimer.
- **The demo numbers.** 75 with one `BANK-WILL-001`, 100 after the fix, back to 75 after undo, 100 for the clean persona.
- **The labels.** Frontend tests pin the null-score wording, the conflict-versus-gap distinction, the disclaimer, the fallback label, the original-quote label and the joint-holding notice — the exact strings the project promised never to misstate.
- **The schema guard.** An outdated database refuses to start rather than failing later at query time.

Three independent semantic reviews during the build returned NEEDS_CHANGES and caught real defects: a stale acknowledgement check, a cross-account NPS leak, supersession decided by file date, a null score rendered as a red zero, a double-counted gap, a 401 with no CORS headers, and a deployment document that overstated what its token gate closed. All were fixed before the work was committed.

13. Deployment, and what is deliberately unfinished

Nothing is deployed. There is no hosted URL, no domain, no TLS and no managed database. What exists is the mechanism for doing it safely, built and verified:

- **An optional shared token.** Set `API_TOKEN` and every route except `/health` requires a bearer header. Off by default, which is correct for a loopback demo. CORS wraps the check, so a rejected browser request shows a real 401 rather than an opaque network failure.
- **A server-side proxy.** Point the browser at `/api/backend` and the Next.js server attaches the token, so the credential never reaches the browser bundle.
- **An honest runbook.** `docs/DEPLOYMENT.md` states that the proxy relays anonymously, so publishing the frontend still leaves the upload path open unless real access control sits in front of it. That is listed as a required step, not an optional hardening.

The honest limitations list

LIMITATION	WHY IT IS ACCEPTABLE HERE
No user accounts or authentication	Loopback-only demo. The token gate exists for hosting but is not user auth.
SQLite with no migrations	All data is synthetic and disposable; a schema change resets it deliberately.
A joint asset extracted as `unknown` is still checked	Most documents do not state a holding pattern. Excluding unknown would silently skip most records, which is worse. Written down in `RULES.md` section 14.
No seed document is a joint holding	`GAP-JOINT-HOLDING` is covered by tests but not clickable in the demo. Adding one would change the headline score of 75.
No institution integration at all	By design. The product needs only documents the user chooses to share.
The settlement checklist is a mock	Labelled **Concept / Future Feature** in the API payload and on its own page, and excluded from `RULES.md` because its steps are not sourced rules.
Frontend tests cover components, not pages	The three tested components carry every safety-critical label; pages are verified by hand.

Where to look next in the repository

FILE	WHY YOU WOULD OPEN IT
`backend/rules/RULES.md`	The authoritative rule set: sources, predicates, exclusions, gap markers.
`backend/app/detection.py`	The engine. Every conflict decision in the product is in this one file.
`DECISIONS.md`	Every architectural decision with its reasoning and what was rejected.
`PROGRESS.md`	Phase-by-phase status, including the one box still open.
`docs/DEPLOYMENT.md`	What hosting would take, and what the token gate does not do.
`reports/`	A completion report per phase, plus the gap-closure report.

JeevanSetu is a hackathon prototype operating on synthetic documents. Not legal advice. The rule set is a limited, India-scoped first pass and is not authoritative; a licensed legal professional and the relevant institution should review any real document.